17 Implementing security in reactive applications

This chapter covers

- Using Spring Security with reactive applications
- Using reactive apps in a system designed using OAuth 2 authentication

Reactive is a programming paradigm where we apply a different way of thinking when developing our applications. Reactive programming is a powerful way of developing web apps that has gained wide acceptance. I would even say that it became fashionable a few years ago, when any important conference had at least several presentations discussing reactive apps. However, like any other technology in software development, reactive programming is not a solution applicable to every situation.

In some cases, a reactive approach is an excellent fit. In other cases, it might only complicate your life. But in the end, the reactive approach exists because it addresses some limitations of imperative programming and is thus used to avoid such limitations. One of them involves executing large tasks that can be fragmented. With an imperative approach, you give the application a task to execute, and the application has the responsibility to solve it. If the task is large, it might take a substantial amount of time for the application to solve it. The client who assigned the task needs to wait for the task to be entirely solved before receiving a response. With reactive programming, you can divide the task so that the app has the opportunity to approach some of the subtasks simultaneously. This way, the client receives the processed data faster.

This chapter discusses application-level security in reactive applications with Spring Security. Like with any other application, security is an important aspect of reactive apps. However, because reactive apps are designed differently, Spring Security has adapted the way we implement features discussed previously in this book.

We'll start with a short overview of implementing reactive apps with the Spring framework in section 17.1. Then we'll apply the security features you learned throughout this book on security apps. In section 17.2, we'll discuss user management in reactive apps, and in section 17.3, we'll continue applying authorization rules. Finally, in section 17.4, you'll learn how to implement reactive applications in a system designed over OAuth 2. You'll learn what changes from the Spring Security perspective when it comes to reactive applications, and of course, you'll learn how to apply this through examples.

17.1 What are reactive apps?

In this section, we briefly discuss reactive apps. This chapter is about applying security for reactive apps, so here, I want to make sure you grasp the essentials of reactive apps before going deeper into Spring Security configurations. Because the topic of reactive applications is big, I only review the main aspects of reactive apps as a refresher. If you aren't yet aware of how reactive apps work, or you need to understand them in more detail, I recommend you read part 3 of *Spring in Action, Sixth Edition*, by Craig Walls (Manning, 2022).

When we implement apps, we use two ways to implement the functionalities. The following list elaborates on these approaches:

- *With the imperative approach, your app processes the bulk of your data all at once.* For example, a client app calls an endpoint exposed by the server and sends all the data that needs to be processed to the backend. Say you implement a functionality where the user uploads files. If the user selects a number of files, and all of these are received by the backend app to be processed at once, you're working with an imperative approach.
- *With the reactive approach, your app receives and processes the data in fragments.* Not all the data has to be fully available from the beginning to be processed. The backend receives and processes data as it gets it. Say the user selects some files, and the backend needs to upload and process them. The backend doesn't wait to receive all the files at once before processing. The backend might receive the files one by one and process each while waiting for more files to come.

Figure 17.1 presents an analogy for the two programming approaches. Imagine a factory bottling milk. If the factory gets all the milk in the morning and delivers the milk once it finishes the bottling, we say it's non-reactive (imperative). If the factory gets the milk throughout the day and delivers the orders once it finishes bottling enough milk, we say it's reactive. Clearly, for the milk factory, it's more advantageous to use a reactive approach rather than a non-reactive one.

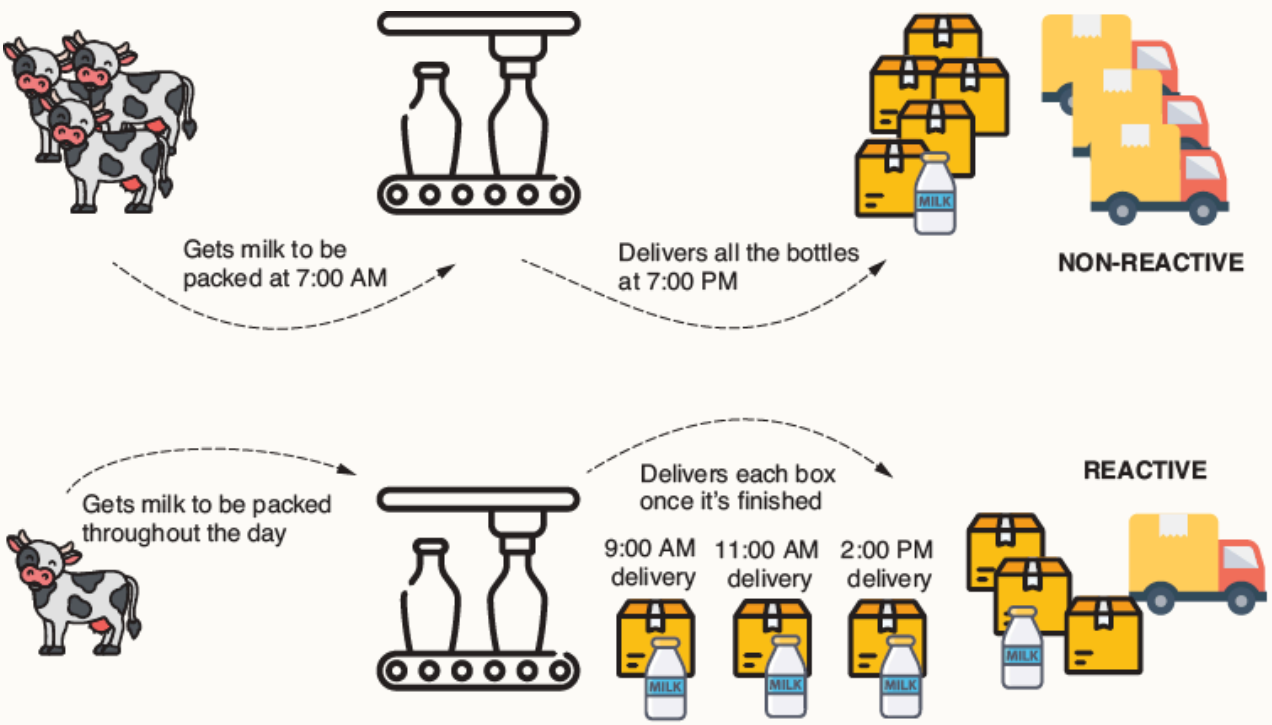


Figure 17.1 Non-reactive vs. reactive. In a non-reactive approach, the milk factory gets all the milk to be packaged in the morning and delivers all the boxes in the evening. In a reactive approach, as the milk is brought to the factory, it's packaged and then delivered. For this scenario, a reactive approach is better as it allows milk to be collected throughout the day and delivered sooner to the clients.

For implementing reactive apps, the Reactive Streams specification (**http://www.reactive-streams.org/**) provides a standard way for asynchronous stream processing. One of the implementations of this specification is the Project Reactor, which builds the foundations of Spring's reactive programming model. Project Reactor provides a functional API for composing Reactive Streams.

To get a more hands-on feeling, let's start a simple implementation of a reactive app. We'll continue further with this same application in section 17.2 when discussing user management in reactive apps. I created a new project named ssia-ch17-ex1, and we'll develop a reactive web app that exposes a demo endpoint. In the pom.xml file, we must add the reactive web dependency as presented in the following code snippet. This dependency houses the Project Reactor and enables us to use its related classes and interfaces in our project:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Next, we define a simple `HelloController` to hold the definition of our demo endpoint. Listing 17.1 shows the definition of the `HelloController` class. In the endpoint definition, you'll observe I used a `Mono` as a return type. `Mono` is one of the essential concepts defined by a Reactor implementation. When working with Reactor, you often use `Mono` and `Flux`, both defining publishers (sources of data). In the Reactive Streams specification, a publisher is described by the `Publisher` interface. This interface describes one of the essential contracts used

with Reactive Streams. The other contract is the `Subscriber`. This contract describes the component consuming the data.

When designing an endpoint that returns something, the endpoint becomes a publisher, so it has to return a `Publisher` implementation. If using Project Reactor, this will be a `Mono` or a `Flux`. `Mono` is a publisher for a single value, while `Flux` is a publisher for multiple values. Figure 17.2 describes these components and the relationships among them.

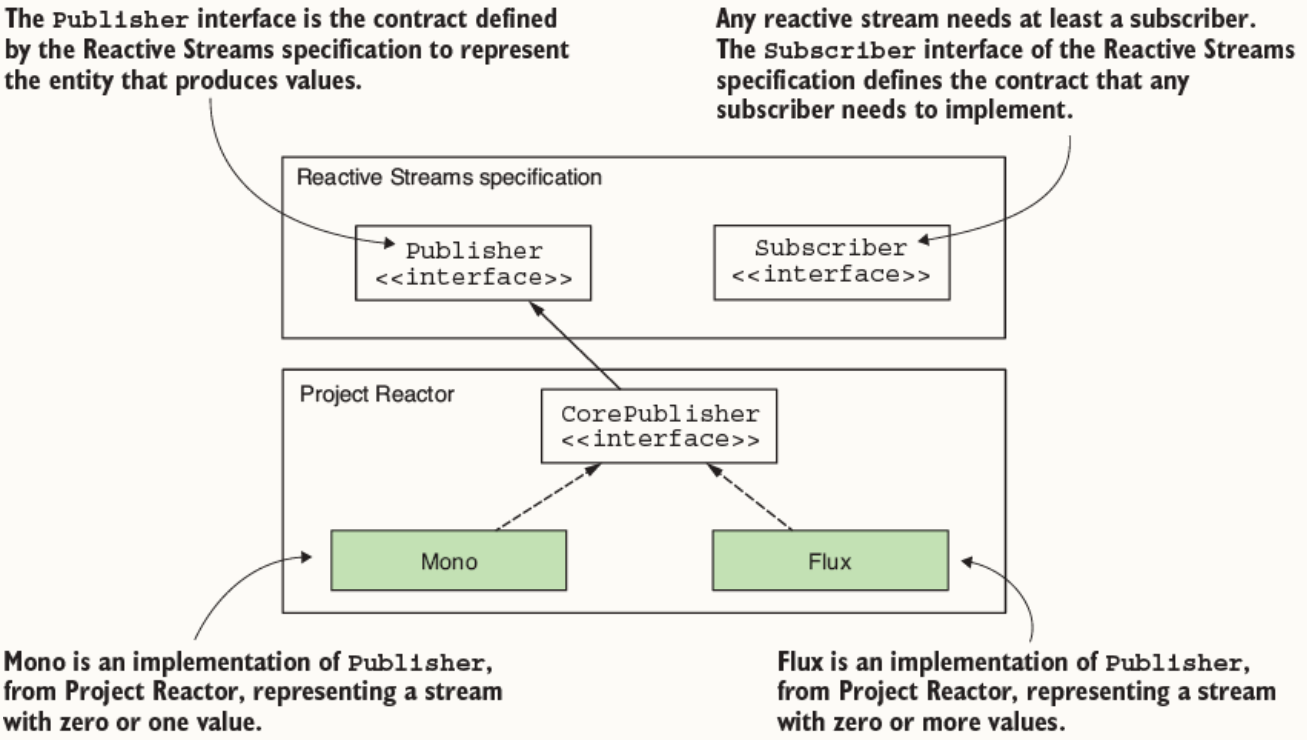


Figure 17.2 In a reactive stream, a publisher produces values, and a subscriber consumes them. Contracts defined by the Reactive Streams specification describe publishers and subscribers. Project Reactor implements the Reactive Streams specification and implements the `Publisher` and `Subscriber` contracts. In the figure, the components we use in the examples in this chapter are shaded.

To make this explanation even more precise, let's go back to the milk factory analogy. The milk factory is a reactive backend implementation that exposes an endpoint to receive the milk to be processed. This endpoint produces something (bottled milk), so it needs to return a `Publisher`. If more than one bottle of milk is requested, then the milk factory needs to return a `Flux`, which is Project Reactor's `Publisher` implementation that deals with zero or more produced values.

Listing 17.1 The definition of the `HelloController` class

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public Mono<String> hello() {
    return Mono.just("Hello!");      ①
```

```
    }
}
```

① Creates and returns a Mono stream source with one value on the stream

You can now start and test the application. The first thing you observe by looking in the app's terminal is that Spring Boot doesn't configure a Tomcat server anymore. Spring Boot used to configure a Tomcat for a web application by default, and you may have observed this aspect in any of the examples developed previously in this book. Instead, Spring Boot now autoconfigures Netty as the default reactive web server for a Spring Boot project.

The second thing you may have observed when calling the endpoint is that it doesn't behave differently from an endpoint developed with a non-reactive approach. You can still find in the HTTP response body the `Hello!` message that the endpoint returns in its defined `Mono` stream. The next code snippet presents the app's behavior when calling the endpoint:

```
curl http://localhost:8080/hello
```

The response body is

```
Hello!
```

But why is the reactive approach different in terms of Spring Security? Behind the scenes, a reactive implementation uses multiple threads to solve the tasks on the stream. In other words, it changes the philosophy of one-thread-per-request, which we use for a web app designed with an imperative approach (figure 17.3). And from here, there are more differences:

- The `SecurityContext` implementation doesn't work the same way in reactive applications. Remember, the `SecurityContext` is based on a `ThreadLocal`, and now we have more than one thread per request.
- Because of the `SecurityContext`, any authorization configuration is now affected. Remember from chapter 5 that the authorization rules generally rely on the `Authentication` instance stored in the `SecurityContext`. Now the security configurations applied at the endpoint layer, as well as the global method security functionality, are affected.
- The `UserDetailsService`, the component responsible for retrieving the user details, is a data source. Because of this, the user details service also needs to support a reactive approach. (We learned about this contract in chapter 2.)

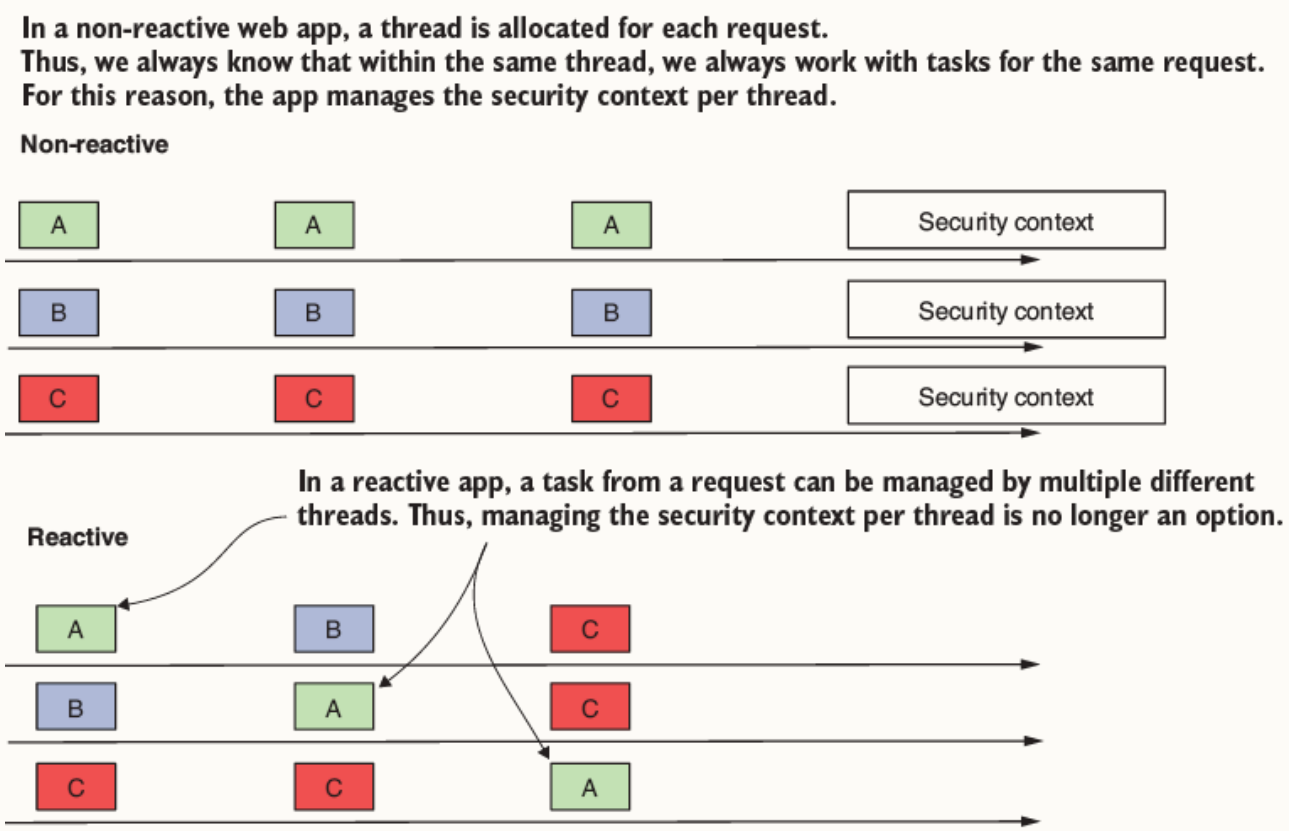


Figure 17.3 In the figure, each arrow represents the timeline of a different thread, and squares represent the processed tasks from requests A, B, and C. Because in a reactive app, tasks from one request might be handled on multiple threads, the authentication details cannot be stored at the thread level anymore.

Figure 17.4 presents another way to see this approach. Imagine a team of people working on a set of tasks. Each can take a task and leave it when they are blocked. It's not always the same thread that will continue the task that has been left over. So the security context can't be addressed to a thread anymore, but it must somehow be linked to the task.

Fortunately, Spring Security offers support for reactive apps and covers all cases where you can't use the implementations for non-reactive apps anymore. We'll continue in this chapter by discussing the way you implement security configurations with Spring Security for reactive apps. We'll start in section 17.2 with implementing user management and continue in section 17.3 with applying endpoint authorization rules, where we'll find out how security context works in reactive apps. We'll then continue our discussion with reactive method security, which replaces the global method security of imperative apps.

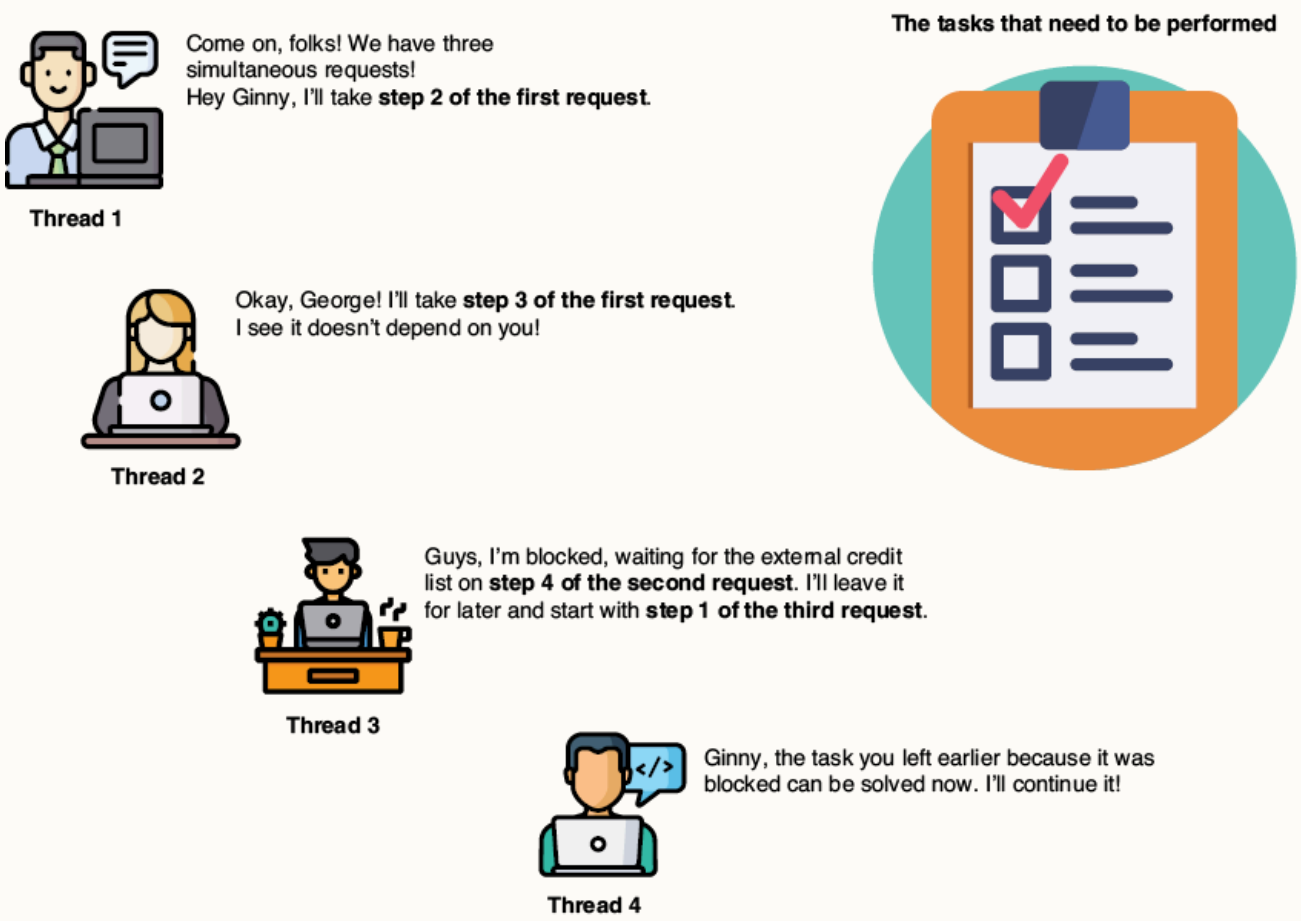


Figure 17.4 An analogy of the way a reactive app works. A thread doesn't take a request's tasks in order and wait when it's blocked. Instead, all tasks from all requests are on a backlog. Any available thread can work on tasks from any request. This way, independent tasks can be solved in parallel, and the threads don't stay idle.

17.2 User management in reactive apps

Often in applications, the way a user authenticates is based on a pair of username and password credentials. This approach is basic, and we've discussed it, starting with the most straightforward application we implemented in chapter 2. But with reactive apps, the implementation of the component taking care of user management changes as well. In this section, we discuss implementing user management in a reactive app.

We continue the implementation of the ssia-ch17-ex1 application we started in section 17.1 by adding a `ReactiveUserDetailsService` to the context of the application. We want to make sure the /hello endpoint can be called only by an authenticated user. As its name suggests, the `ReactiveUserDetailsService` contract defines the user details service for a reactive app.

The definition of the contract is as simple as the one for `UserDetailsService`. The `ReactiveUserDetailsService` defines a method used by Spring Security to retrieve a user by its username. The difference is that the method described by the `ReactiveUserDetailsService` directly returns a `Mono<UserDetails>` and not the `UserDetails` like for `UserDetailsService`. The next code snippet shows the definition of the `ReactiveUserDetailsService` interface:

```
public interface ReactiveUserDetailsService {
  Mono<UserDetails> findByUsername(String username);
```

```
}
```

As in the case of the `UserDetailsService`, you can write a custom implementation of the `ReactiveUserDetailsService` to give Spring Security a way to obtain the user details. To simplify this demonstration, we use an implementation provided by Spring Security. The `MapReactiveUserDetailsService` implementation stores the user details in memory (the same as with the `InMemoryUserDetailsManager` that you learned about in chapter 2). We change the pom.xml file of the ssia-ch17-ex1 project and add the Spring Security dependency, as the next code snippet shows:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

We then create a configuration class and add a `ReactiveUserDetailsService` and a `PasswordEncoder` to the Spring Security context. I named the configuration class `ProjectConfig`. You can find the definition of this class in listing 17.2. Using a `ReactiveUserDetailsService`, we define one user with its username `john`, the password `12345`, and an authority I named `read`. As you can observe, it's similar to working with a `UserDetailsService`. The main difference in the implementation of the `ReactiveUserDetailsService` is that the method returns a reactive `Publisher` object containing the `UserDetails` instead of the `UserDetails` instance itself. Spring Security takes the rest of the duty of integration.

Listing 17.2 The `ProjectConfig` class

```
@Configuration
public class ProjectConfig {

 @Bean                                          ①
 public ReactiveUserDetailsService userDetailsService() {
  var  u = User.withUsername("john")                    ②
         .password("12345")
         .authorities("read")
         .build();

  var uds = new MapReactiveUserDetailsService(u);          ③

  return uds;
```

```
  }

  @Bean                                          ④
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① Adds a ReactiveUserDetailsService to the Spring context

② Creates a new user with its username, password, and authorities

③ Creates a MapReactiveUserDetailsService to manage the UserDetails instances

④ Adds a PasswordEncoder to the Spring context

Now, when starting and testing the application, you might notice that you can call the endpoint only when you authenticate using the proper credentials. In our case, we can only use `john` with its password `12345`, as it's the only user record we added. The following code snippet shows the behavior of the app when calling the endpoint with valid credentials:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello!
```

Figure 17.5 explains the architecture we use in this application. Behind the scenes, an `AuthenticationWebFilter` intercepts the HTTP request. This filter delegates the authentication responsibility to an authentication manager. The authentication manager implements the `ReactiveAuthenticationManager` contract. Unlike non-reactive apps, we don't have authentication providers. The `ReactiveAuthenticationManager` directly implements the authentication logic.

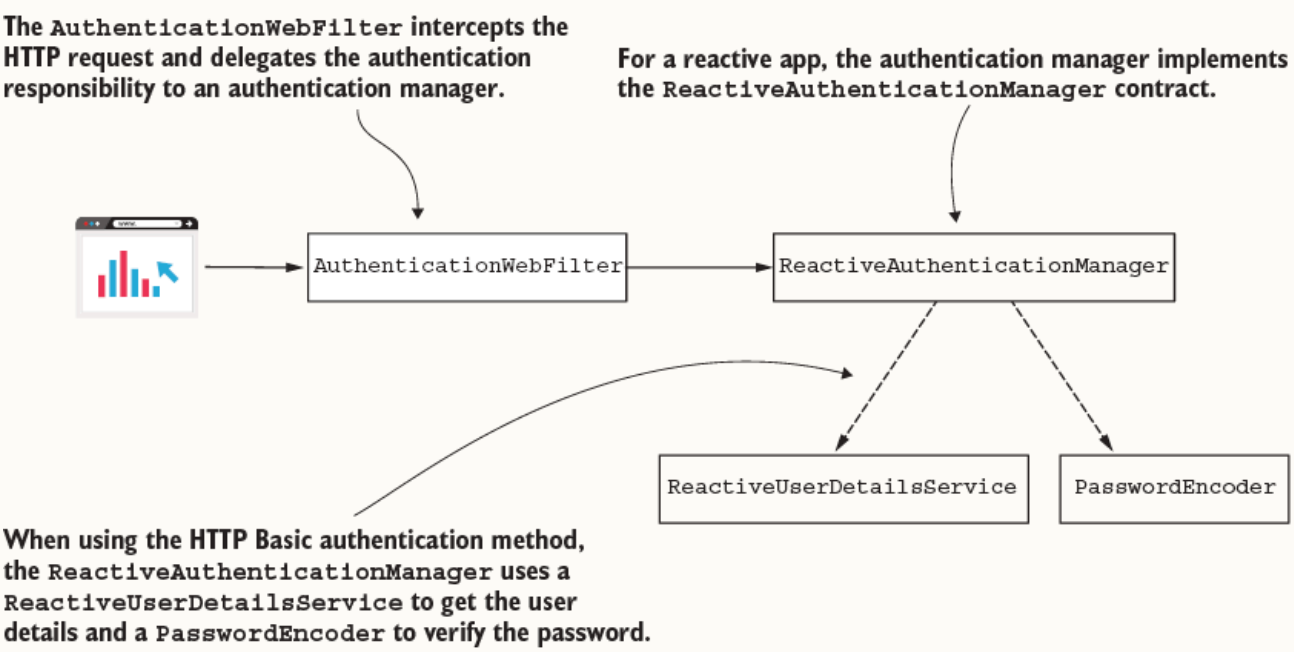


Figure 17.5 An `AuthenticationWebFilter` intercepts the request and delegates the authentication responsibility to a `ReactiveAuthenticationManager`. If the authentication logic involves users and passwords, the `ReactiveAuthenticationManager` uses a `ReactiveUserDetailsService` to find the user details and a `PasswordEncoder` to verify the password.

If you want to create your own custom authentication logic, implement the `ReactiveAuthenticationManager` interface. The architecture for reactive apps is not much different from the one for non-reactive applications that we already discussed throughout this book. As presented in figure 17.4, if authentication involves user credentials, then we use a `ReactiveUserDetailsService` to obtain the user details and a `PasswordEncoder` to verify the password.

Moreover, the framework still knows to inject an authentication instance when you request it. You request the `Authentication` details by adding `Mono<Authentication>` as a parameter to the method in the controller class. Listing 17.3 presents the changes done to the controller class. Again, the significant change is that you use reactive publishers. Observe that we need to use `Mono<Authentication>` instead of the plain `Authentication` like we did in non-reactive apps.

Listing 17.3 The `HelloController` class

```
@RestController
public class HelloController {

 @GetMapping("/hello")
 public Mono<String> hello(
  Mono<Authentication> auth) {     ①

  Mono<String> message =           ②
   auth.map(a -> "Hello " + a.getName());

  return message;
```

```
    }
}
```

① Requests the framework to provide the authentication object

② Returns the name of the principal in the response

Rerunning the application and calling the endpoint, you observe the behavior is as presented in the next code snippet:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello john
```

And now you're probably wondering, Where did the `Authentication` object come from? Being that this is a reactive app, we can't afford to use a `ThreadLocal` anymore because the framework is designed to manage the `SecurityContext`. But Spring Security offers us a different implementation of the context holder for reactive apps, `ReactiveSecurityContextHolder`. We use this to work with the `SecurityContext` in a reactive app. So we still have the `SecurityContext`, but now it's managed differently. Figure 17.6 describes the end of the authentication process once the `ReactiveAuthenticationManager` successfully authenticates the request.

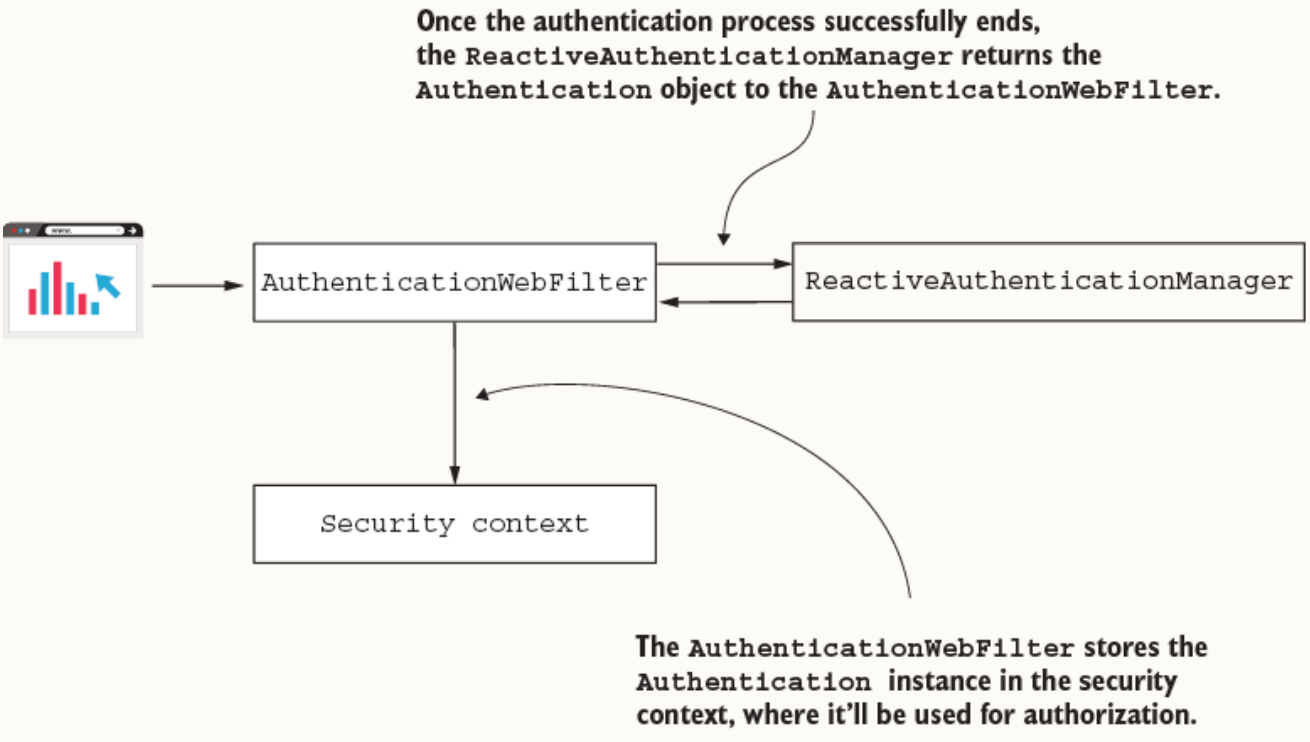


Figure 17.6 Once the `ReactiveAuthenticationManager` successfully authenticates the request, it returns the `Authentication` object to the filter. The filter stores the `Authentication` instance in the `SecurityContext`.

Listing 17.4 shows you how to rewrite the controller class if you want to get the authentication details directly from the security context. This approach is an alternative to allowing the framework to inject it through the method's parameter. You'll find this change implemented in project ssia-ch17-ex2.

Listing 17.4 Working with a `ReactiveSecurityContextHolder`

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public Mono<String> hello() {
    Mono<String> message =
      ReactiveSecurityContextHolder.getContext()    ①

        .map(ctx -> ctx.getAuthentication())        ②

        .map(auth -> "Hello " + auth.getName());    ③

    return message;
  }
}
```

① From the ReactiveSecurityContextHolder, it takes a Mono<SecurityContext>.

② Maps the SecurityContext to the Authentication object

③ Maps the Authentication object to the returned message

If you rerun the application and test the endpoint again, you can observe that it behaves the same as in the previous examples of this section. Here's the command:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello john
```

Now that you know Spring Security offers an implementation to properly manage the `SecurityContext` in a reactive environment, you know this is how your app applies the authorization rules. And these details that you just learned open the way to configuring the authorization rules, which we'll discuss in section 17.3.

17.3 Configuring authorization rules in reactive apps

In this section, we discuss configuring authorization rules. As you already know from the previous chapters, authorization follows authentication. We discussed in sections 17.1 and 17.2 how Spring Security manages users and the `SecurityContext` in reactive apps. But once the app finishes authentication and stores the details of the authenticated request in the `SecurityContext`, it's time for authorization.

As for any other application, you probably need to configure authorization rules when developing reactive apps as well. To teach you how to set authorization rules in reactive apps, we'll discuss first in section 17.3.1 the way you make configurations at the endpoint layer. Once we finish discussing authorization configuration at the endpoint layer, you'll learn in section 17.3.2 how to apply it at any other layer of your application using method security.

17.3.1 Applying authorization at the endpoint layer in reactive apps

In this section, we discuss configuring authorization at the endpoint layer in reactive apps. Setting the authorization rules in the endpoint layer is the most common approach for configuring authorization in a web app. You already discovered this while working on the previous examples in this book. Authorization configuration at the endpoint layer is essential—you use it in almost every app. Thus, you need to know how to apply it for reactive apps as well.

You learned from previous chapters to set the authorization rules by adding a bean of type `SecurityFilterChain` to the app's context. This approach doesn't work in reactive apps. To teach you how to configure authorization rules for the endpoint layer properly for reactive apps, we start by working on a new project, which I named ssia-ch17-ex3.

In reactive apps, Spring Security uses a contract named `SecurityWebFilterChain` to apply the configurations we used to do by using a bean of type `SecurityFilterChain`, as discussed in previous chapters. With reactive apps, we add a bean of type `SecurityWebFilterChain` in the Spring context. To teach you how to do this, let's implement a basic application having two endpoints that we secure independently. In the pom.xml file of our newly created ssia-ch17-ex3 project, add the dependencies for reactive web apps and, of course, Spring Security:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Create a controller class to define the two endpoints for which we configure the authorization rules. These endpoints are accessible at the paths /hello and /ciao. To call the /hello endpoint, a user needs to authenticate, but you can call the /ciao endpoint without authentication. The following listing presents the definition of the controller.

Listing 17.5 The `HelloController` class defining the endpoints to secure

```
@RestController
public class HelloController {

 @GetMapping("/hello")
 public Mono<String> hello(Mono<Authentication> auth) {
  Mono<String> message = auth.map(a -> "Hello " + a.getName());
  return message;
 }

 @GetMapping("/ciao")
 public Mono<String> ciao() {
  return Mono.just("Ciao!");
 }
}
```

In the configuration class, we make sure to declare a `ReactiveUserDetailsService` and a `PasswordEncoder` to define a user, as you learned in section 17.2. The following listing defines these declarations.

Listing 17.6 The configuration class declaring components for user management

```
@Configuration
public class ProjectConfig {

 @Bean
 public ReactiveUserDetailsService userDetailsService() {
  var  u = User.withUsername("john")
        .password("12345")
        .authorities("read")
        .build();

  var uds = new MapReactiveUserDetailsService(u);

  return uds;
 }

 @Bean
 public PasswordEncoder passwordEncoder() {
  return NoOpPasswordEncoder.getInstance();
```

```
  }

  // ...
}
```

In listing 17.7, we work in the same configuration class we declared in listing 17.6, but omit the declaration of the `ReactiveUserDetailsService` and the `PasswordEncoder` so that you can focus on the authorization configuration we discuss. In listing 17.7, you might notice that we add a bean of type `SecurityWebFilterChain` to the Spring context. The method receives as a parameter an object of type `ServerHttpSecurity`, which is injected by Spring. `ServerHttpSecurity` enables us to build an instance of `SecurityWebFilterChain`. `ServerHttpSecurity` provides methods for configuration similar to the ones you used when configuring authorization for non-reactive apps.

Listing 17.7 Configuring endpoint authorization for reactive apps

```
@Configuration
public class ProjectConfig {

  // Omitted code

  @Bean
  public SecurityWebFilterChain securityWebFilterChain(
    ServerHttpSecurity http) {

    http.httpBasic(Customizer.withDefaults());

    http.authorizeExchange(                    ①
      c -> c.pathMatchers(HttpMethod.GET, "/hello")    ②
            .authenticated()                   ③
          .anyExchange()                       ④
            .permitAll()                       ⑤
    );


    return http.build();                       ⑥
  }
}
```

① Begins the endpoint authorization configuration

② Selects the requests for which we apply the authorization rules

③ Configures the selected requests to only be accessible when authenticated

④ Refers to any other request

⑤ Allows requests to be called without needing authentication

⑥ Builds the SecurityWebFilterChain object to be returned

We start the authorization configuration with the `authorizeExchange()` method. We call this method similarly to the way we call the `authorizeHttpRequests()` method when configuring endpoint authorization for non-reactive apps. Then we continue by using the `pathMatchers()` method. You can consider this method as the equivalent of using `requestMatchers()` when configuring endpoint authorization for non-reactive apps.

As for non-reactive apps, once we use the matcher method to group requests to which we apply the authorization rule, we then specify what the authorization rule is. In our example, we called the `authenticated()` method, which states that only authenticated requests are accepted. You also used a method named `authenticated()` when configuring endpoint authorization for non-reactive apps. The methods for reactive apps are named the same to make them more intuitive. Similarly to the `authenticated()` method, you can also call these methods:

- `permitAll()` —Configures the app to allow requests without authentication
- `denyAll()` —Denies all requests
- `hasRole()` *and* `hasAnyRole()` —Apply rules based on roles
- `hasAuthority()` *and* `hasAnyAuthority()` —Apply rules based on authorities

It looks like something's missing, doesn't it? Do we also have an `access()` method like we had for configuring authorization rules in non-reactive apps? Yes. But it's a bit different, so we'll work on a separate example to prove it. Another similarity in naming is the `anyExchange()` method that takes the role of what used to be `anyRequest()` in non-reactive apps.

NOTE Why is it called `anyExchange()`, and why didn't the developers keep the same name for the method `anyRequest()`? Why `authorizeExchange()` and why not `authorizeHttpRequests()`? This difference stems from the terminology used with reactive apps. We generally refer to communication between two components in a reactive fashion as *exchanging data*. This reinforces the image of data being sent as segmented in a continuous stream and not as a big bunch in one request.

We also need to specify the authentication method like any other related configuration. We do this with the same `ServerHttpSecurity` instance, using methods with the same name and in the same fashion you learned to use for non-reactive apps: `httpBasic()`, `formLogin()`, `csrf()`, `cors()`, adding filters and customizing the filter chain, and so on. In the end, we call the `build()` method to create the instance of `SecurityWebFilterChain`, which we finally return to add to the Spring context.

I told you earlier in this section that you can also use the `access()` method in the endpoint authorization configuration of reactive apps just as you can for non-reactive apps. But as I said when discussing the configuration of non-reactive apps in chapters 7 and 8, use the `access()` method only when you can't apply your configuration otherwise. The `access()` method offers you great flexibility, but also makes your app's configuration more difficult to read. Always prefer the simpler solution over the more complex one. However, you'll find situations in which you need this flexibility. For example, suppose you have to apply a more complex authorization rule, and using `hasAuthority()` or `hasRole()` and its companion methods isn't enough. For this reason, I'll also teach you how to use the `access()` method. I created a new project named ssia-ch17-ex4 for this example. In the next listing, you can see how I built the `SecurityWebFilterChain` object to allow access to the /hello path only if the user has the admin role. Also, access can be allowed only before noon. For all other endpoints, I restrict access completely.

Listing 17.8 Using the `access()` method when implementing configuration rules

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public SecurityWebFilterChain
  securityWebFilterChain(ServerHttpSecurity http) {

  http.httpBasic(Customizer.withDefaults());

  http.authorizeExchange(
   c -> c.anyExchange()                          ①
         .access(this::getAuthorizationDecisionMono)
  );


  return http.build();
 }

 private Mono<AuthorizationDecision>
  getAuthorizationDecisionMono(                  ②
        Mono<Authentication> a,
        AuthorizationContext c) {

  String path = getRequestPath(c);               ③

  boolean restrictedTime =
   LocalTime.now().isAfter(LocalTime.NOON);

  if(path.equals("/hello")) {                    ④
```

```
      return  a.map(isAdmin())
            .map(auth -> auth && !restrictedTime)
            .map(AuthorizationDecision::new);
      }

      return Mono.just(new AuthorizationDecision(false));
    }

    // Omitted code
  }
```

① For any request, it applies a custom authorization rule.

② The method defining the custom authorization rule receives the Authentication and the request context as parameters.

③ From the context, it obtains the path of the request.

④ For the /hello path, it applies the custom authorization rule.

It might look difficult, but it's not that complicated. When you use the `access()` method, you provide a function receiving all possible details about the request, which are the `Authentication` object and the `AuthorizationContext`. Using the `Authentication` object, you have the details of the authenticated user: username, roles or authorities, and other custom details depending on how you implement the authentication logic. The `AuthorizationContext` provides the information on the request: the path, headers, query params, cookies, and so on.

The function you provide as a parameter to the `access()` method should return an object of type `AuthorizationDecision`. As you guessed, `AuthorizationDecision` is the answer that tells the app whether the request is allowed. When you create an instance with `new AuthorizationDecision(true)`, it means that you allow the request. If you create it with `new AuthorizationDecision(false)`, it means you disallow the request.

In listing 17.9, you find the two methods I omitted in listing 17.8 for your convenience: `getRequestPath()` and `isAdmin()`. By omitting these, I let you focus on the logic used by the `access()` method. As you can observe, the methods are simple. The `isAdmin()` method returns a function that returns true for an `Authentication` instance having the `ROLE_ADMIN` attribute. The `getRequestPath()` method simply returns the path of the request.

Listing 17.9 The definition of the `getRequestPath()` and `isAdmin()` methods

```
@Configuration
public class ProjectConfig {
```

```
  // Omitted code

  private String getRequestPath(AuthorizationContext c) {
    return c.getExchange()
          .getRequest()
          .getPath()
          .toString();
  }

  private Function<Authentication, Boolean> isAdmin() {
    return p ->
      p.getAuthorities().stream()
       .anyMatch(e -> e.getAuthority().equals("ROLE_ADMIN"));
  }
}
```

Running the application and calling the endpoint either result in a response status 403 Forbidden if any of the authorization rules we applied aren't fulfilled or simply display a message in the HTTP response body:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello john
```

What happened behind the scenes in the examples in this section? When authentication ended, another filter intercepted the request. The `AuthorizationWebFilter` delegates the authorization responsibility to a `ReactiveAuthorizationManager` (figure 17.7).

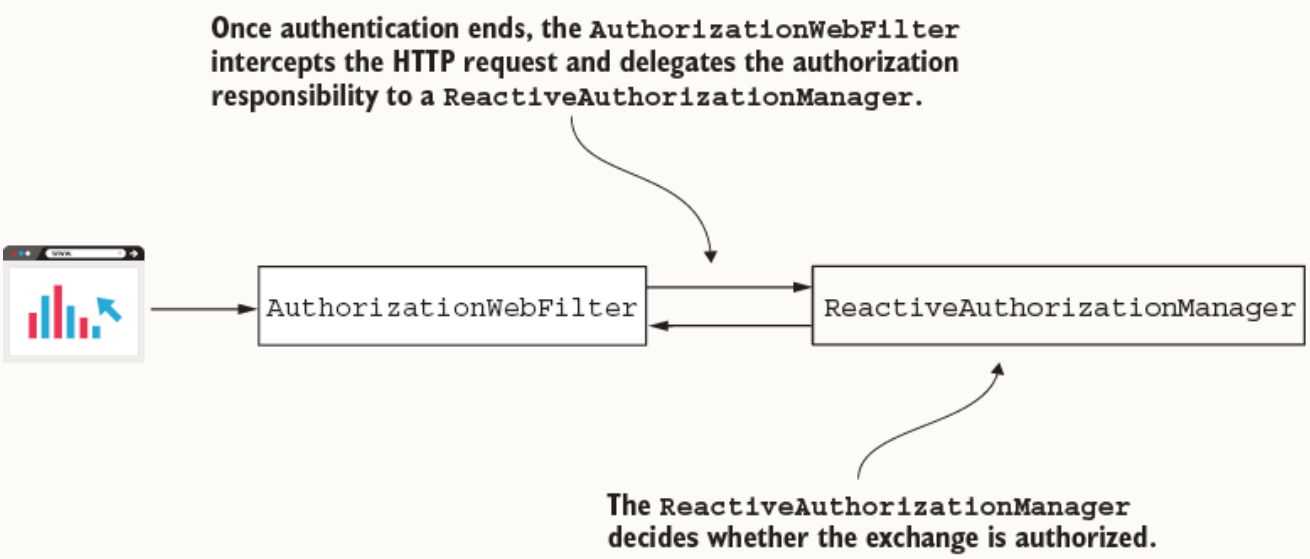


Figure 17.7 After the authentication process successfully ends, another filter, named `AuthorizationWebFilter`, intercepts the request. This filter delegates the authorization responsibility to a `ReactiveAuthorizationManager`.

Wait! Does this mean we only have a `ReactiveAuthorizationManager`? How does this component know how to authorize a request based on the configurations we made? To answer the first question: no, there are actually multiple implementations of the `ReactiveAuthorizationManager`. The `AuthorizationWebFilter` uses the `SecurityWebFilterChain` bean we added to the Spring context. With this bean, the filter decides which `ReactiveAuthorizationManager` implementation to delegate the authorization responsibility to (figure 17.8).

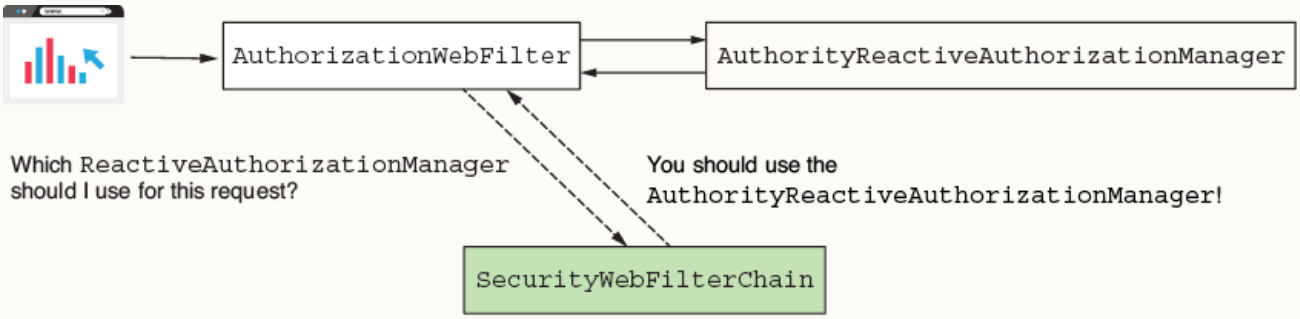


Figure 17.8 The `AuthorizationFilter` uses the `SecurityWebFilterChain` bean (shaded) that we added to the context to know which `ReactiveAuthorizationManager` to use.

17.3.2 Using method security in reactive apps

In this section, we discuss applying authorization rules for all layers of reactive apps. For non-reactive apps, we used method security, and in chapters 11 and 12, you learned different approaches to applying authorization rules at the method level. Being able to apply authorization rules at layers other than the endpoint layer offers you great flexibility and enables you to apply authorization for non-web applications. To teach you how to use method security for reactive apps, we work on a separate example, which I named ssia-ch17-ex5.

Instead of global method security, when working with non-reactive apps, we call the approach *reactive* method security, where we apply authorization rules directly at the method level. For our example, we use `@PreAuthorize` to validate that a user has a specific role to call a test endpoint. To keep the example simple, we use the `@PreAuthorize` annotation directly over the method defining the endpoint. But you can use it the same way we discussed in chapters 11 and 12 for non-reactive apps: on any other component method in your reactive application. Listing 17.10 shows the definition of the controller class. Observe that we use `@PreAuthorize`, similar to what you learned in chapter 11. Using SpEL expressions, we declare that only an admin can call the annotated method.

Listing 17.10 The definition of the controller class

```
@RestController
public class HelloController {

 @GetMapping("/hello")
 @PreAuthorize("hasRole('ADMIN')")      ①
 public Mono<String> hello() {
  return Mono.just("Hello");
 }
}
```

① Uses @PreAuthorize to restrict access to the method

Here you find the configuration class in which we use the annotation `@EnableReactiveMethodSecurity` to enable the reactive method security feature. Similar to the method security, we need to explicitly use an annotation to enable it. Besides this annotation, in the `configuration` class, you also find the usual user management definition.

Listing 17.11 The `configuration` class

```
@Configuration
@EnableReactiveMethodSecurity      ①
public class ProjectConfig {

 @Bean
 public ReactiveUserDetailsService userDetailsService() {
  var u1 = User.withUsername("john")
       .password("12345")
       .roles("ADMIN")
       .build();

  var u2 = User.withUsername("bill")
       .password("12345")
       .roles("REGULAR_USER")
```

```
        .build();

    var uds = new MapReactiveUserDetailsService(u1, u2);

    return uds;
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① Enables the reactive method security feature

You can now start the application and test the behavior of the endpoint by calling it for each user. You should observe that only John can call the endpoint because we defined him as the admin. Bill is just a regular user, so if we try to call the endpoint authenticating as Bill, we get back a response with the status HTTP 403 Forbidden. Calling the /hello endpoint authenticating with user John looks like this:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello!
```

Calling the /hello endpoint authenticating with user Bill looks like this:

```
curl -u bill:12345 http://localhost:8080/hello
```

The response body is

```
Access Denied
```

Behind the scenes, this functionality works the same as for non-reactive apps. In chapters 11 and 12, you learned that an aspect intercepts the call to the method and implements the authorization. If the call doesn't fulfill the specified preauthorization rules, the aspect doesn't delegate the call to the method (figure 17.9).

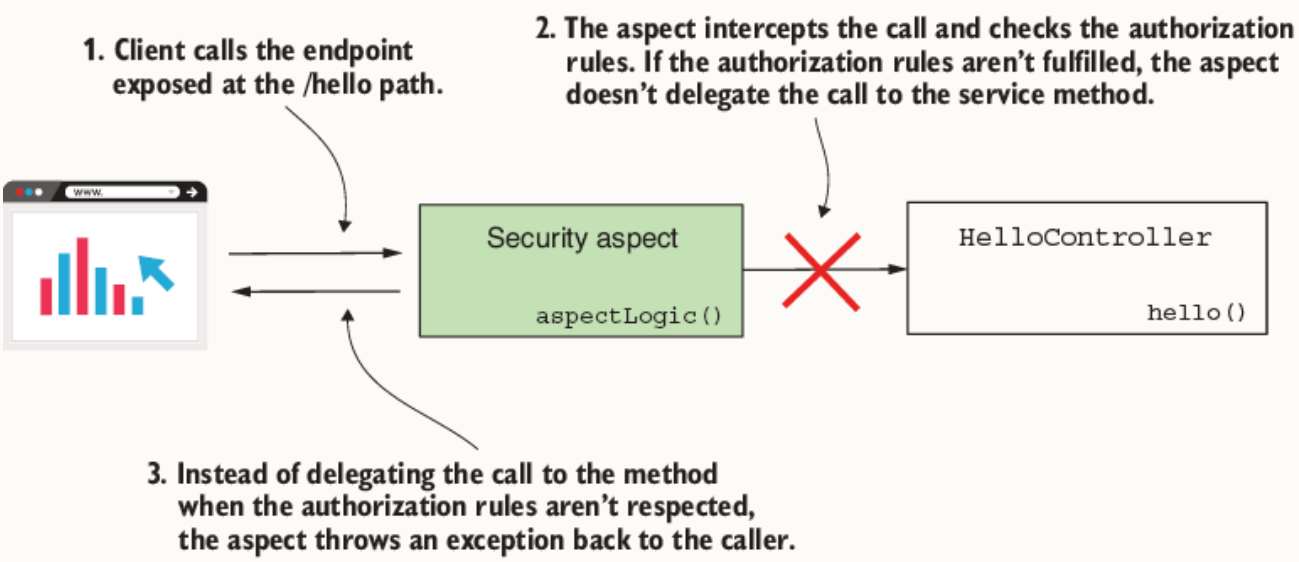


Figure 17.9 When using method security, an aspect intercepts the call to a protected method. If the call doesn't fulfill the preauthorization rules, the aspect doesn't delegate the call to the method.

17.4 Creating a reactive OAuth 2 resource server

You're probably wondering by now if we could use reactive applications in a system designed over the OAuth 2 framework. In this section, we discuss implementing a resource server as a reactive app. You learn how to configure your reactive application to rely on an authentication approach implemented over OAuth 2. Because using OAuth 2 is so common nowadays, you might encounter requirements where your resource server application needs to be designed as a reactive server. I created a new project named ssia-ch17-ex6 , and we'll implement a reactive resource server application. You need to add the dependencies in pom.xml, as the next code snippet illustrates:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-oauth2</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

We need an endpoint to test the application, so we add a controller class. The next code snippet presents the controller class:

```
@RestController
public class HelloController {

 @GetMapping("/hello")
 public Mono<String> hello() {
  return Mono.just("Hello!");
 }
}
```

And now the most important part of the example: the security configuration. For this example, we configure the resource server to use the public key exposed by the authorization server for token signature validation.

To configure the authentication method, we use the `SecurityWebFilterChain`, as you learned about in section 17.3. However, instead of using the `httpBasic()` method, we call the `oauth2ResourceServer()` method. Then, by calling the `jwt()` method, we define the kind of token we use, and by using a `Customizer` object, we specify the way the token signature is validated. In the next listing, you can find the definition of the `configuration` class.

Listing 17.12 Defining the security web filter chain configuration

```
@Configuration
public class ProjectConfig {

 @Value("${jwk.endpoint}")
 private String jwkEndpoint;

 @Bean
 public SecurityWebFilterChain securityWebFilterChain(
  ServerHttpSecurity http) {

  http.oauth2ResourceServer(                ①
     c -> c.jwt(
       j -> j.jwkSetUri(jwkEndpoint);    ②
     )
  );

  http.authorizeExchange(
    c -> c.anyExchange().authenticated()

  );

  return http.build();
 }
}
```

① Configures the resource server authentication method

② Specifies the way the token is validated

In the same way, we could've configured the public key instead of specifying an URI where the public key is exposed. The only change was to call the `publicKey()` method of the `jwtSpec` instance and provide a valid public key as a parameter. You can use any of the approaches we discussed in chapter 13, where we analyzed in detail approaches for the resource server to validate the access token.

Next, we change the application.properties file to add the value for the URI where the key set is exposed, as well as change the server port to 9090. This way, we allow the authorization server to run on 8080. In the next code snippet, you'll find the contents of the application.properties file:

```
server.port=9090
jwk.endpoint=http://localhost:8080/
➥auth/realms/master/protocol/openid-connect/certs
```

Let's run and prove that the app has the expected behavior that we want. We generate an access token using the authorization server:

```
curl -XPOST 'http://localhost:8080/auth/
➥realms/master/protocol/openid-connect/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=password' \
--data-urlencode 'username=bill' \
--data-urlencode 'password=12345' \
--data-urlencode 'client_id=fitnessapp' \
--data-urlencode 'scope=fitnessapp'
```

In the HTTP response body, we receive the access token as presented here:

```
{
   "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI...",
   "expires_in": 6000,
   "refresh_expires_in": 1800,
   "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5c... ",
   "token_type": "bearer",
   "not-before-policy": 0,
   "session_state": "610f49d7-78d2-4532-8b13-285f64642caa",
   "scope": "fitnessapp"
}
```

Using the access token, we call the /hello endpoint of our application like this:

```
curl -H 'Authorization: BeareryJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJMSE9zTOVRSmJuTmJVb
jhQbVpYQTlUVW9QNTZoWU9OYzNWT2swa1V2ajVVIn...' \
'http://localhost:9090/hello'
```

The response body is

```
Hello!
```

Summary

- Reactive applications have a different style for processing data and exchanging messages with other components. Reactive apps might be a better choice in some situations, like when we can split the data into separate smaller segments for processing and exchanging.
- Like with any other application, you also need to protect reactive apps by using security configurations. Spring Security offers an excellent set of tools you can use to apply security configurations for reactive apps, as well as for non-reactive ones.
- To implement user management in reactive apps with Spring Security, we use the `ReactiveUserDetailsService` contract. This component has the same purpose as `UserDetailsService` for non-reactive apps: it tells the app how to get the user details.
- To implement the endpoint authorization rules for a reactive web application, you need to create an instance of type `SecurityWebFilterChain` and add it to the Spring context. You create the `SecurityWebFilterChain` instance by using the `ServerHttpSecurity` builder.
- Generally, the names of the methods you use to define the authorization configurations are the same as for the methods you use for non-reactive apps. However, you'll find minor naming differences related to the reactive terminology. For example, instead of using `authorizeHttpRequests()`, the name of its counterpart for reactive apps is `authorizeExchange()`.
- Spring Security also provides a way to define authorization rules at the method level, called reactive method security, and it offers great flexibility in applying the authorization rules at any layer of a reactive app. It is similar to what we call global method security for non-reactive apps.

Previous chapter

< Part 5. Going reactive

Next chapter

Part 6. Testing security configurations >